# Huffman Encoding

LECTURE 23

# Huffman Encoding:

- Huffman code is method for the compression of standard text documents.
- It makes use of a binary tree to develop codes of varying lengths for the letters used in the original message.
- Huffman code is also part of the JPEG image compression scheme.
- The algorithm was introduced by David Huffman in 1952 as part of a course assignment at MIT

# Huffman Encoding:

**"traversing threaded binary trees"**

- We have four words including spaces. There is a new line character in the end. The total characters in the phrase are 33 including the space.
- Binary codes are used for alphabets and other language characters. For English alphabets, we use ASCII codes.
- It normally consists of eight bits.
- We can represent lower case alphabets, upper case alphabets, 0,1,…9 and special symbols like $, ! etc.
- In English, you have 26 lower case and 26 upper case alphabets. If you have seen the ASCII table, there are some printable characters and some unprintable characters in it. There are some graphic characters also in the ASCII table.

# Huffman Encoding:

- In our example we have 33 characters.
- If each character take 8 bits in ASCII.
- Then how many bits we need to transfer? It will be 33x8=264.
- But the use of Huffman algorithm can help send the same message with only 116 bits.
- So we can save around 40% using the Huffman algorithm.

# Huffman Encoding:

- List all the letters used, including the "space" character, along with the frequency with which they occur in the message.
- Consider each of these (character, frequency) pairs to be nodes; they are actually leaf nodes.
- Pick the two nodes with the lowest frequency. If there is a tie, pick randomly amongst those with equal frequencies.
- Make a new node out of these two, and turn two nodes into its children.
- This new node is assigned the sum of the frequencies of its children.
- Continue the process of combining the two nodes of lowest frequency till the time only one node, the root is left.

# Huffman Encoding:

- List all the letters used, including the "space" character, along with the frequency with which they occur in the message.
- Consider each of these (character, frequency) pairs to be nodes; they are actually leaf nodes.
- Pick the two nodes with the lowest frequency. If there is a tie, pick randomly amongst those with equal frequencies.
- Make a new node out of these two, and turn two nodes into its children.
- This new node is assigned the sum of the frequencies of its children.
- Continue the process of combining the two nodes of lowest frequency till the time only one node, the root is left.

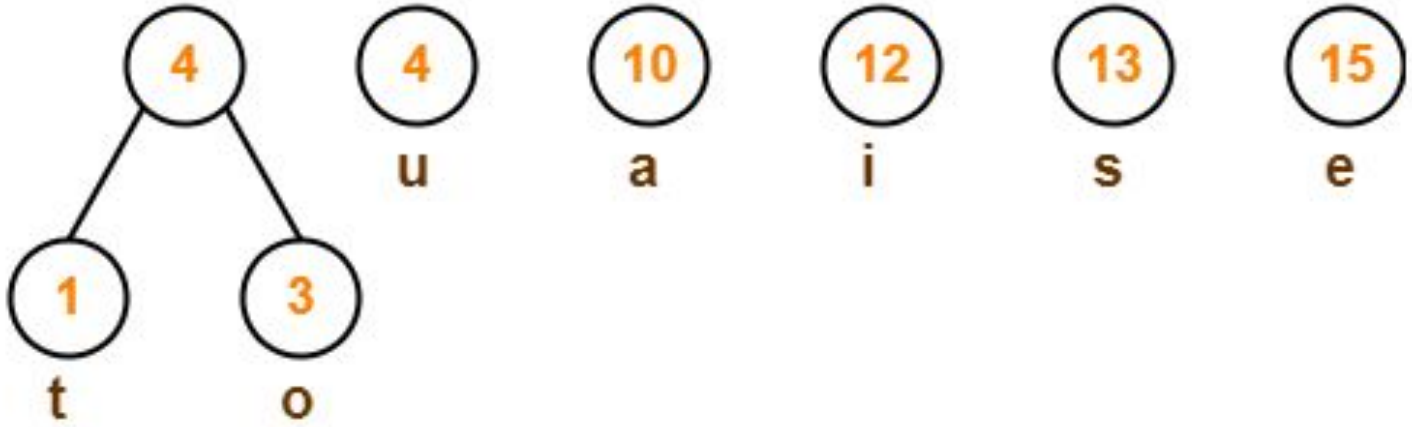
# Huffman Encoding:

- The code length of a character depends on how frequently it occurs in the given text.
- The character which occurs most frequently gets the smallest code.
- The character which occurs least frequently gets the largest code.

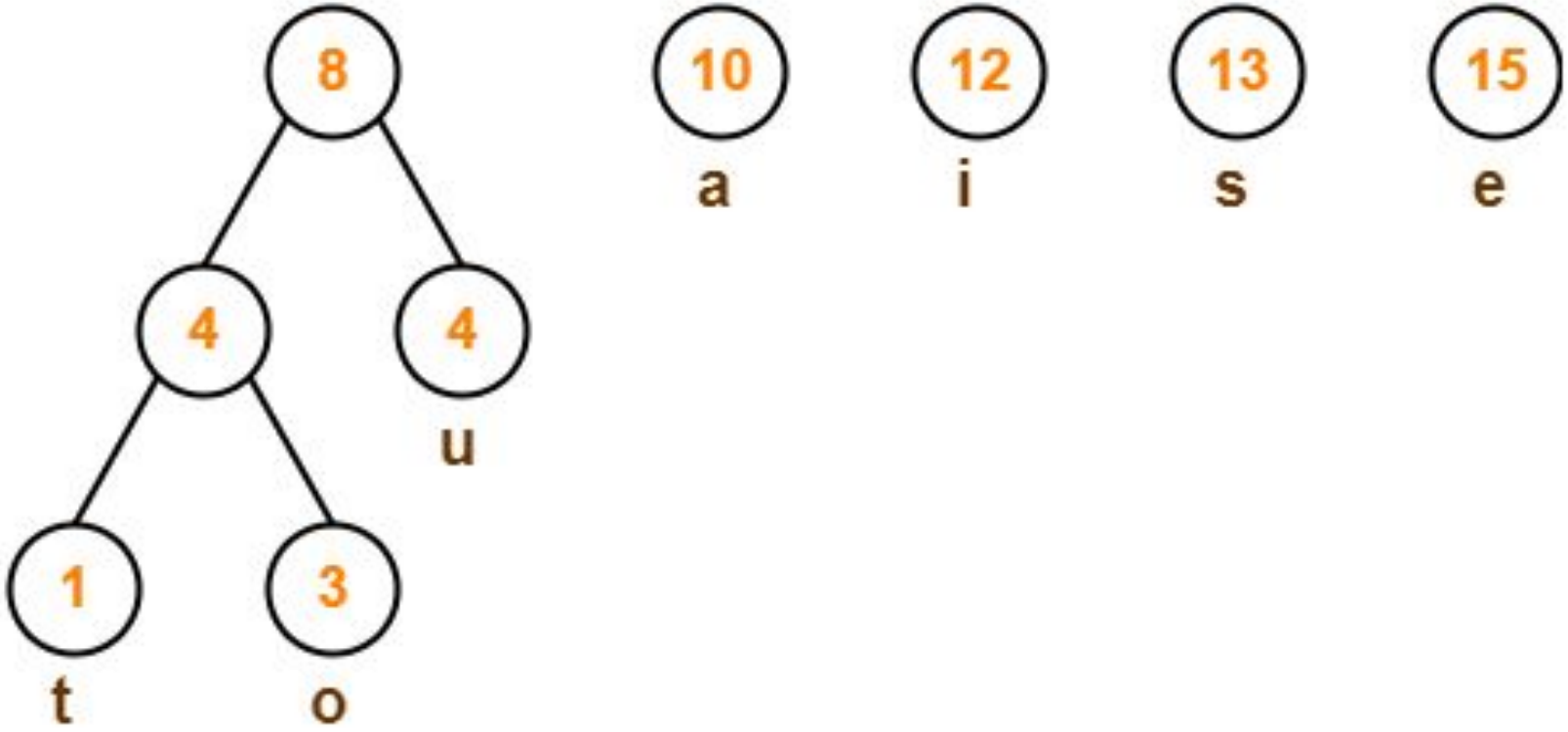
| Characters | Frequencies |
|---|---|
| a | 10 |
| e | 15 |
| i | 12 |
| o | 3 |
| u | 4 |
| s | 13 |
| t | 1 |

**Step-01:**

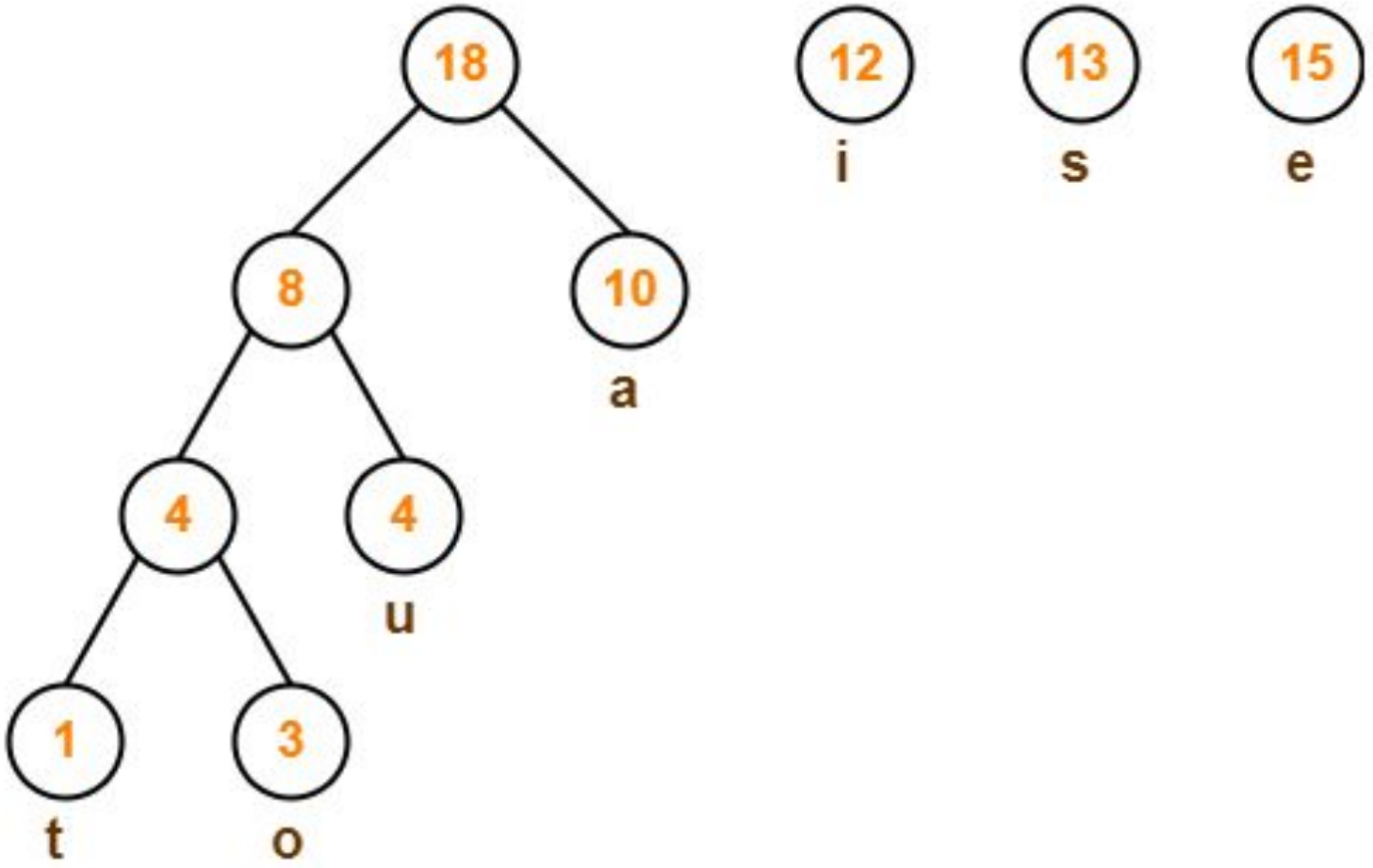

**Step-02:**



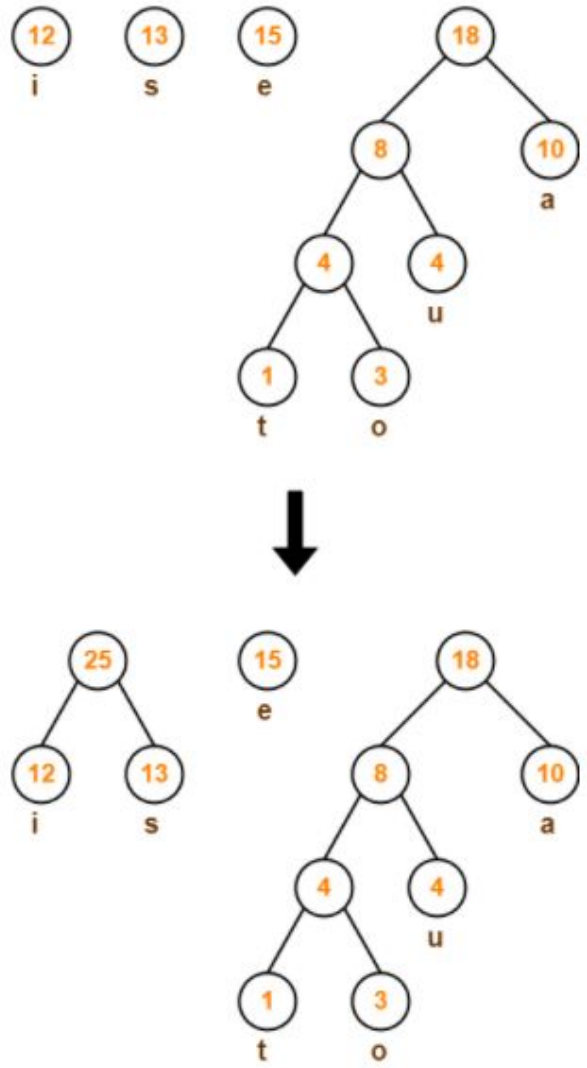
**Step-03:**



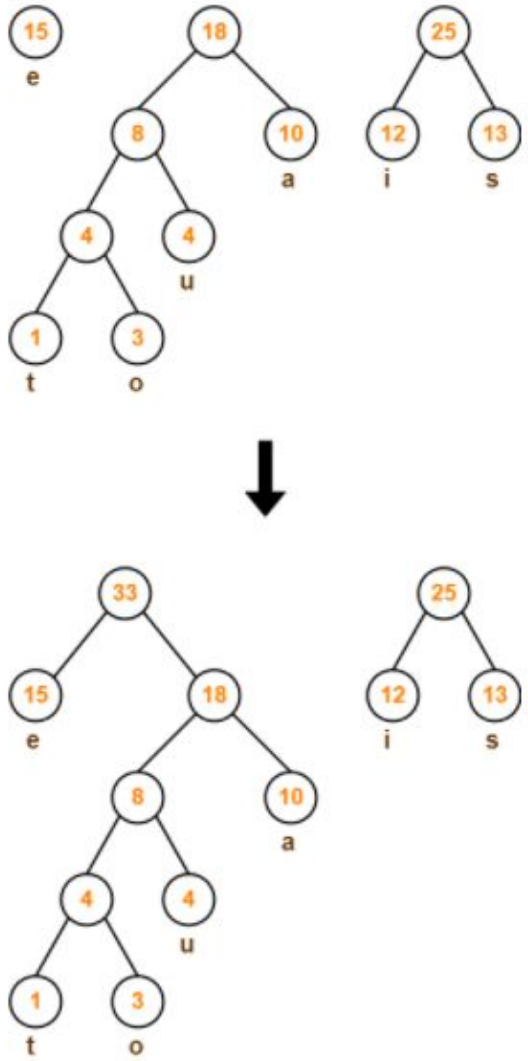
**Step-04:**



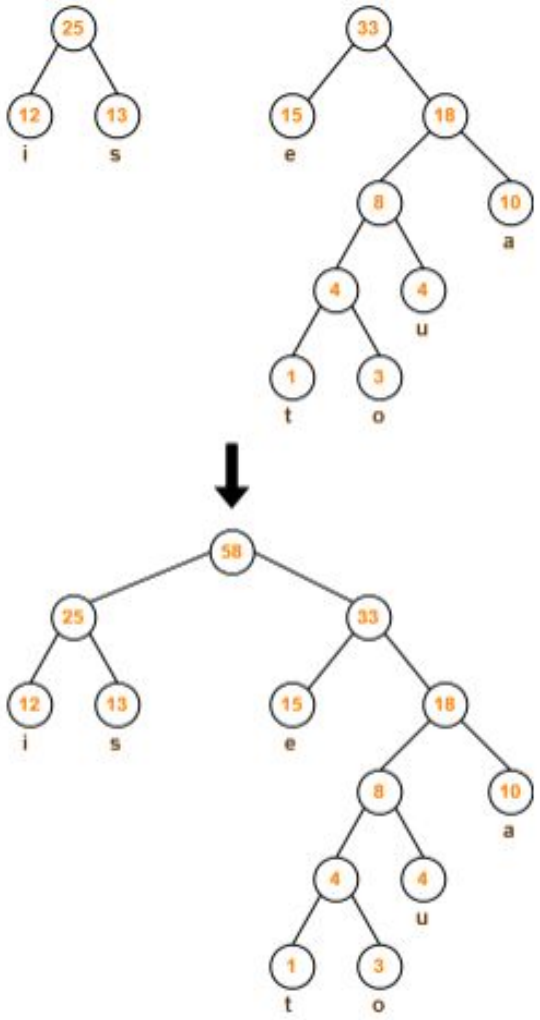
**Step-05:**



**Step-06:**


15
e
18
8
10
a
4
4
u
1
3
t
o
25
12
13
i
s
33
15
e
18
8
10
a
4
4
u
1
3
t
o
25
12
13
i
s

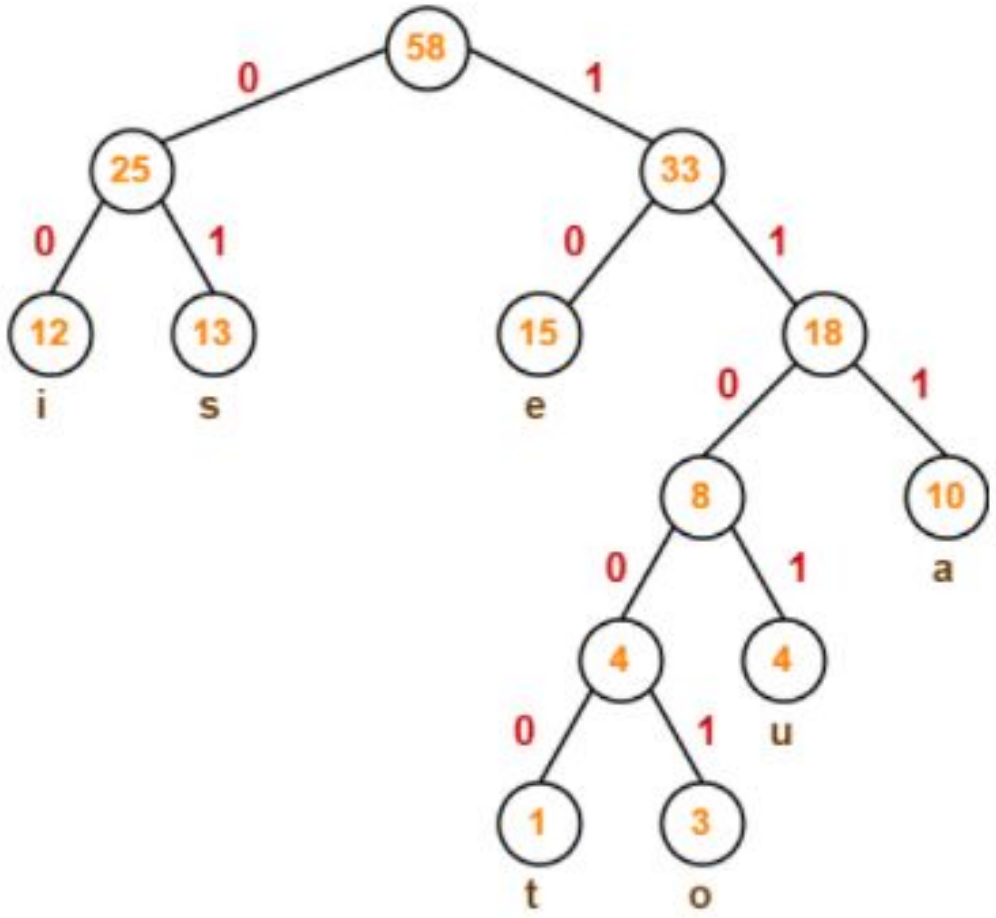
Step-07:




Huffman Tree

# Huffman Encoding:

- We assign weight to all the edges of the constructed Huffman Tree.
- Let us assign weight ‘0’ to the left edges and weight ‘1’ to the right edges.
- If you assign weight ‘0’ to the left edges, then assign weight ‘1’ to the right edges.
- If you assign weight ‘1’ to the left edges, then assign weight ‘0’ to the right edges.
- Any of the above two conventions may be followed.
- But follow the same convention at the time of decoding that is adopted at the time of encoding.




Huffman Tree

# Huffman Encoding:

- To write Huffman Code for any character, traverse the Huffman Tree from root node to the leaf node of that character.
- Following this rule, the Huffman Code for each character is-
  - a = 111
  - e = 10
  - i = 00
  - o = 11001
  - u = 1101
  - s = 01
  - t = 11000

# Huffman Encoding:

- Average code length

= ∑ ( frequencyi x code lengthi ) / ∑ ( frequencyi )

= { (10 x 3) + (15 x 2) + (12 x 2) + (3 x 5) + (4 x 4) + (13 x 2) + (1 x 5) } / (10 + 15 + 12 + 3 + 4 + 13 + 1)

= 2.52

- Total number of bits in Huffman encoded message

= Total number of characters in the message x Average code length per character

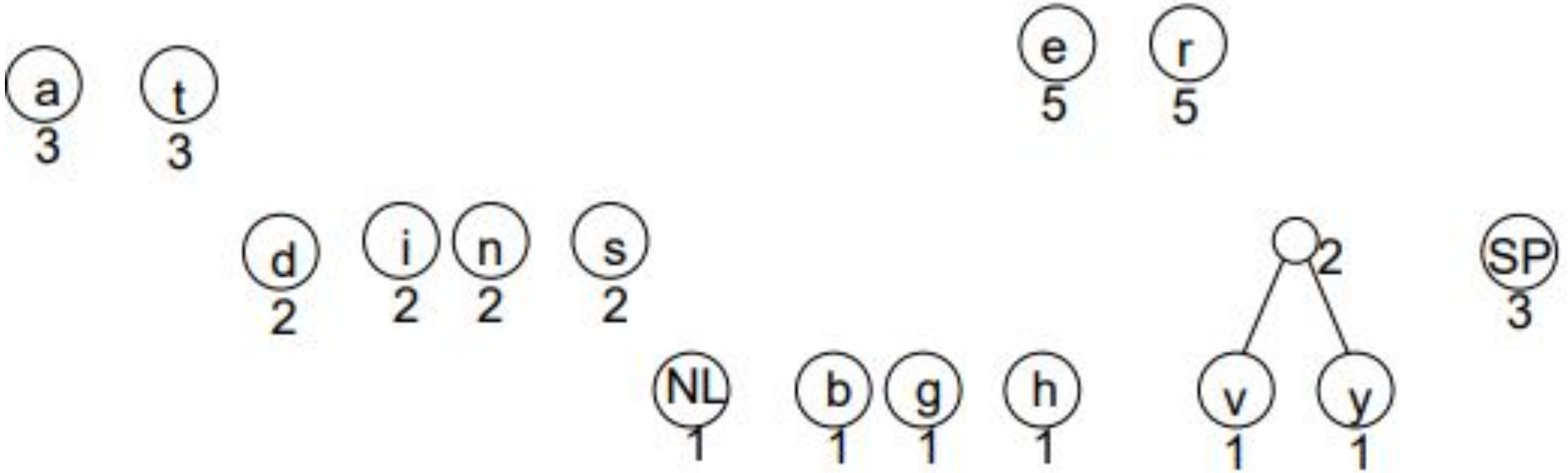
= 58 x 2.52

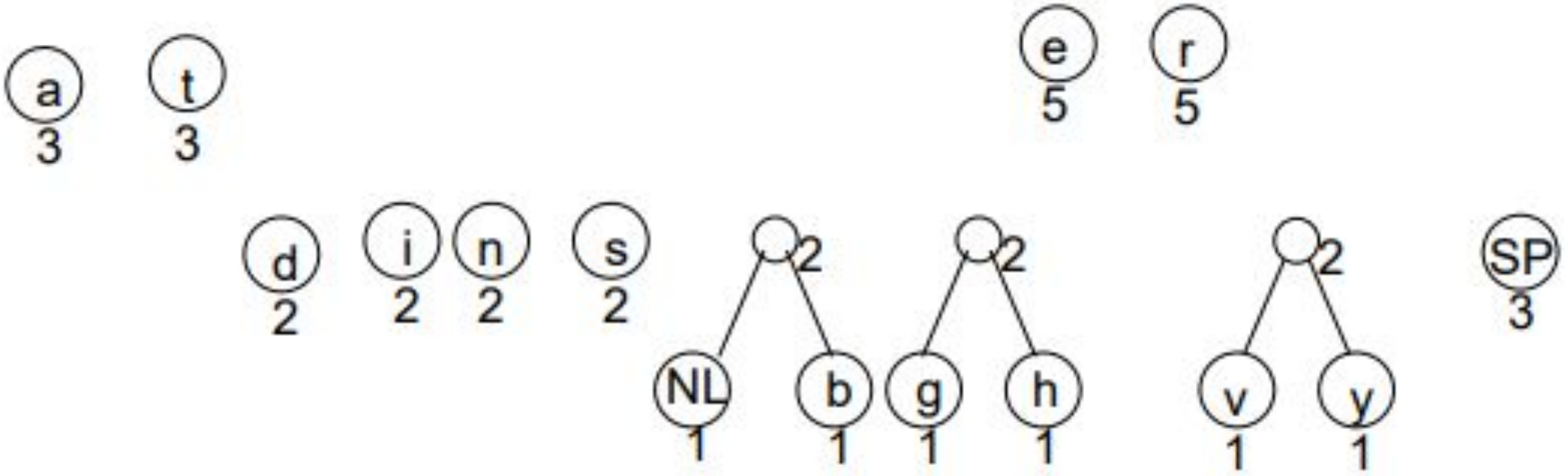
= 146.16

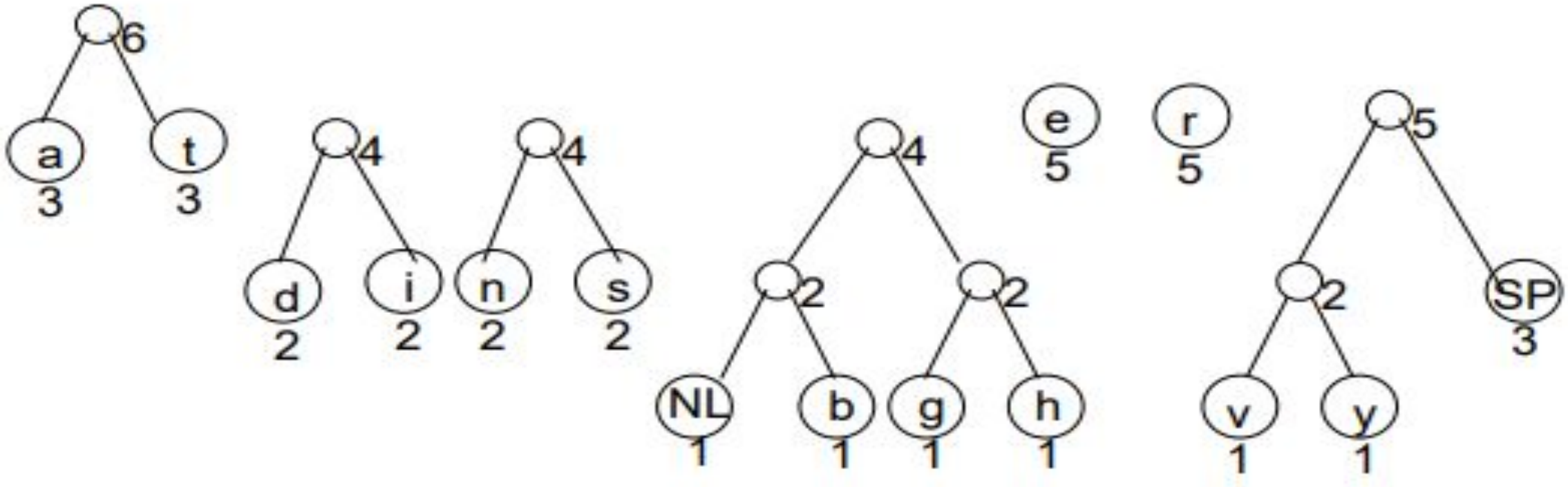
≅ 147 bits

"traversing threaded binary trees"

| Character | frequency | character | frequency |
|---|---|---|---|
| NL | 1 | I | 2 |
| SP | 3 | n | 2 |
| A | 3 | r | 5 |
| B | 1 | s | 2 |
| D | 2 | t | 3 |
| E | 5 | v | 3 |
| G | 1 | y | 1 |
| H | 1 | | |


a
3
t
3
e
5
r
5
d
2
i
2
n
2
s
2
2
SP
3
NL
1
b
1
g
1
h
1
v
1
y
1


a
3
t
3
e
5
r
5
d
2
i
2
n
2
s
2
2
NL
1
b
1
2
g
1
h
1
2
v
1
y
1
2
SP
3




**Complete it by yourself…**

| Character | Code |
|---|---|
| NL | 10000 |
| SP | 1111 |
| a | 000 |
| b | 10001 |
| d | 0100 |
| e | 101 |
| g | 10010 |
| h | 10011 |
| i | 0101 |
| n | 0110 |
| r | 110 |
| s | 0111 |
| t | 001 |
| v | 11100 |
| y | 11101 |